Programa de Pós-Graduação em Ciência da Computação – CIn/UFPE
Visão Computacional – 2025.2
Prof. Carlos Alexandre Barros de Mello (cabm@cin.ufpe.br)
Projeto final
Data de Entrega: até o MEIO-DIA de 15 de dezembro de 2025

OBSERVAÇÕES:

1) **LEMBRE DE COLOCAR SEU NOME**.

2) As respostas devem vir na forma de **relatório**, com explicação, imagens e códigos comentados.

3) As questões podem ser feitas em qualquer ordem. **Suas respostas devem ser escritas com clareza**, ou seja, suas ideias devem estar dispostas de forma clara, organizada e bom português.

4) O arquivo com a solução de ser enviado do seu e-mail do CIn para **cabm@cin.ufpe.br** até o dia/hora estabelecidos. Após esse prazo, qualquer envio será desconsiderado. Organize tudo **em um PDF único**. Lembrem de checar se tudo está nítido. Qualquer material complementar deve ser enviado junto (como imagens geradas).

5) Enumerem as páginas para que a sequência das soluções possa ser observada sem dificuldade.

6) As questões estão descritas de forma bem clara. Qualquer dúvida quanto às questões apenas (não as respostas) devem ser tiradas **APENAS** por e-mail direto para o professor.

7) **TUDO deve estar bem explicado e justificado.**

8) Cópias resultarão em nota **ZERO**.

9) A resolução deve ser enviada **UMA VEZ** apenas. Re-submissões não serão aceitas.

10) **Códigos podem ser apresentados em links, MAS precisam estar registrados e explicados no documento com a solução.**

11) A menos que esteja especificado, sua solução deve resolver o problema apenas para as imagens dadas. Não são soluções gerais que estão sendo pedidas.

12) **Em suas soluções, você deve usar apenas conteúdos vistos na disciplina. Técnicas de pré (ou pós) processamento de imagens podem ser usadas, <u>considerando a revisão dada</u>.**

Questões:

1. Notadamente, espera-se que os sistemas de visão computacional consigam resolver problemas (de natureza visual) que sejam difíceis para o sistema visual humano (SVH). Alguns desses problemas são bastante clássicos e se caracterizam por limitações do SVH que não se espera encontrar em sistemas computacionais. Com essa ideia, resolva, computacionalmente, os problemas apresentados, a seguir, **nas letras a, b e c**. Em todos os casos, explique sua ideia e apresente o código. Todas as imagens citadas estão disponíveis no arquivo compactado disponibilizado junto com o projeto.

a) O segmento de reta em vermelho na imagem abaixo segue até o segmento azul ou o verde? Essa imagem, em tons de cinza, está disponível no arquivo `illusion.png`. A borda da figura está colocada apenas para melhor visualização. Implemente um programa que responda automaticamente a essa pergunta. Sua saída pode ser da forma de uma imagem (deixando claro ao usuário a continuação do segmento de reta), ou como uma resposta ("segmento de baixo" ou "segmento de cima"). É importante apenas que não gere dúvida ao usuário.

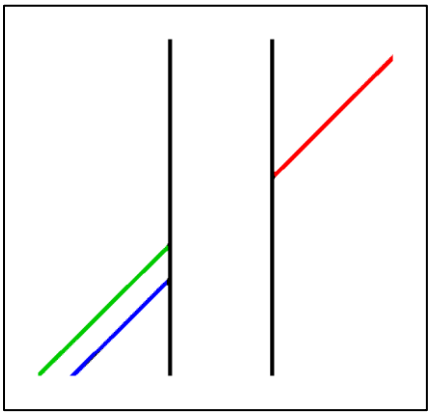

Figura 1. Imagem da Questão 1a: illusion.png.

b) Implemente um programa que mostre automaticamente que os segmentos de reta horizontais na imagem do arquivo `zollner_illusion.png` **são paralelos**. Esse é o efeito conhecido como Ilusões de Zollner. Observe que a questão pede para indicar se os segmentos são paralelos ou não; não é para detectá-los apenas.

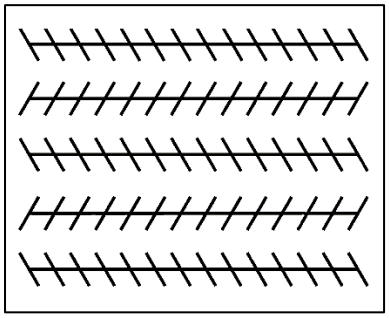

Figura 2. Imagem da Questão 1b: zollner_illusion.png.

c) Implemente um programa que mostre automaticamente que os círculos na imagem do arquivo `circle_illusion.png` têm o mesmo raio (ou diâmetro). Não precisa calcular esse raio (ou diâmetro).

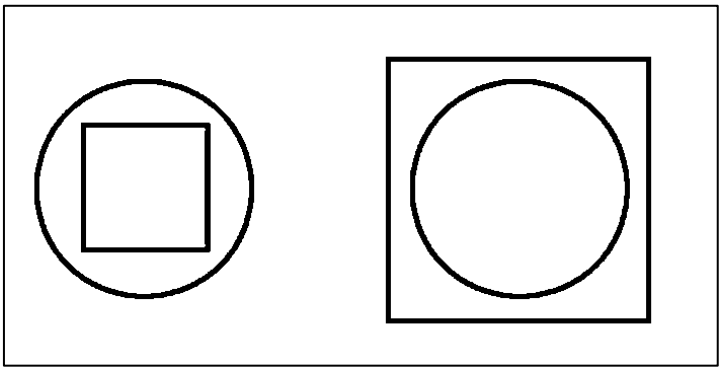

Figura 3. Imagem da Questão 1c: circle_illusion.png.

2. Na imagem da Figura 4, implemente um programa que localize automaticamente a placa de sinalização de proibido estacionar mais próxima.

Figura 4. Imagem da Questão 2: placas.png.

3. Considere duas classes de texturas: classe1 e classe2. As imagens `text1_1.png`, `text1_2.png` e `text1_3.png` pertencem à classe 1, enquanto as imagens `text2_1.png`, `text2_2.png` e `text2_3.png` pertencem à classe 2. Implemente um programa que classifique automaticamente as imagens de cada classe, sem confusão. Você deve usar as imagens 1 e 2 de cada classe para ajustar seu método e deve testá-lo com as imagens 3 de cada classe. O método não pode confundir as classes.

Você não pode usar mais imagens além dessas que foram dadas.

4. Considere a imagem `tomates.png`. Implemente um programa que detecte automaticamente os quatro tomates da imagem (tanto os vermelhos quanto os verdes). Seu programa deve retornar **onde** esses tomates estão na imagem ou como uma resposta visual ou como coordenadas (centro e raio). **Por exemplo**, pode ser como na imagem da Figura 5-direita. O tomate ocluso pode ser ignorado. Mas observe que os tomates maduros e verdes foram detectados.

Figura 5. (esquerda) Imagem da Questão 4 e (direita) possível forma de saída.

5. Considere as imagens `Buildings.png`, `Church.png`, `ColorChecker.png`, `toysflah.png` e `toysnoflash.png`. Todas as imagens têm, nitidamente, algum problema nas cores. Implemente os algoritmos de White Patch Retinex e Gray World e verifique os resultados gerados em cada uma das imagens. Quais foram os melhores resultados aparentes? Quais os piores? **Disserte sobre os resultados.**

OBS: A questão a seguir deve ser resolvida por todos que tiraram menos de 4,0 (quatro) na avaliação. Ela não será considerada para os outros alunos. Se resolvida corretamente, ela será usada para complementar a nota da prova para, junto com as outras questões deste projeto, poder gerar um conceito C. Ela **não substituirá a nota da prova** e a aprovação do aluno dependerá da resolução correta de **todas** as outras questões do projeto.

6. Considere a imagem a seguir à esquerda ("Entrada"). Suponha que A, B e C são paredes (as letras são apenas ilustrativas, não fazendo parte da cena real) em um ambiente fechado. Desenvolva um algoritmo (parte de um sistema autônomo com visão por máquina) que receba essa imagem de entrada e consiga atribuir automaticamente uma legenda que distinga a parede C. Considere condições ideais e constantes de iluminação. O sistema deve funcionar como na Figura 6: a imagem da direita sendo a saída do sistema (considerando como verde a legenda atribuída à parede – a cor é irrelevante). O sistema não conhece a imagem previamente, processando-a da forma como está na "Entrada". **Não precisa implementar.**

Seu algoritmo deve funcionar em qualquer situação semelhante a essa (ou seja, não apenas nessa imagem, mas nessas condições). Você está buscando paredes laterais que tendem para o fundo do ambiente (como C, nas Figuras 6a e 6b). As partes superior e inferior das imagens são o teto e o chão, respectivamente. A Figura 6b é outro exemplo onde o algoritmo deve funcionar da mesma forma (essa figura é apenas a Figura 6a espelhada). Nas imagens, as alturas das paredes A e C, nos cantos esquerdo e direito, respectivamente, são as mesmas.

Sua solução precisa usar algum conceito de percepção visual.

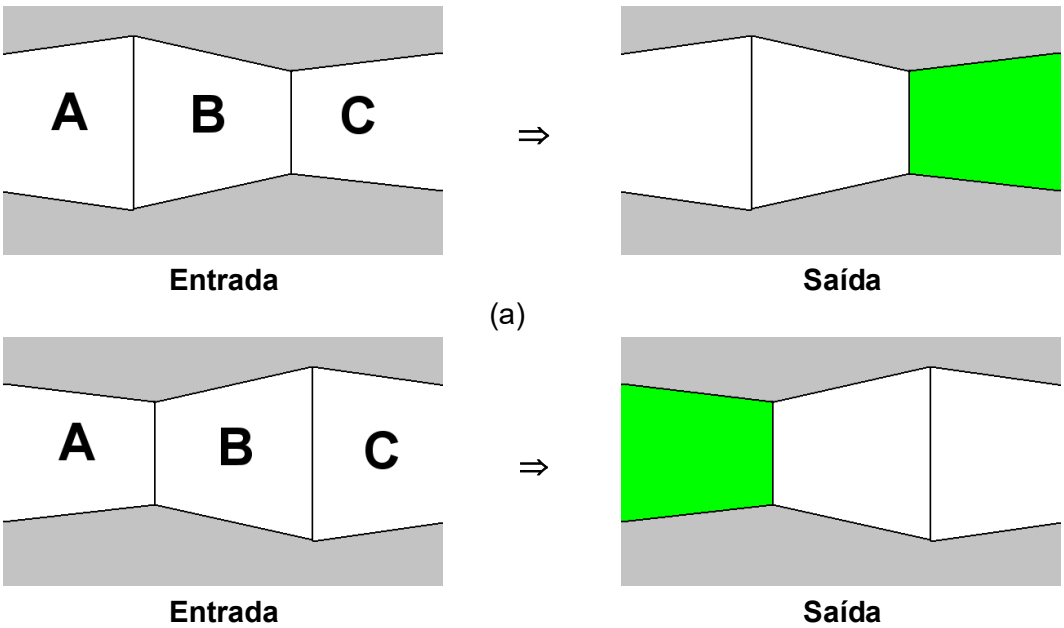

Figura 6. Imagem da Questão 6.